EAG Four-Role RAG Cognitive Agent: Architecture and System Design

Varun Sood

School of AI — Extensive AI Agents (EAG V3 Session 7)

Abstract—This paper describes the architecture and implementation of a cognitive agent that decomposes multi-step reasoning into four specialized roles—Memory, Perception, Decision, and Action—rather than relying on a monolithic prompt. Session 7 extends the design with FAISS vector memory, document indexing, and retrieval-augmented generation (RAG) over course papers and a 1,500-item AI jobs corpus. The orchestrator (`agent7.py`) executes a bounded loop (maximum 24 iterations) until Perception marks all goals complete. All large language model (LLM) inference and embeddings route through the course LLM Gateway V7; twelve Model Context Protocol (MCP) tools run in a separate stdio server. Design principles enforce separation of concerns, intent-only planning in Perception, single-goal Decision steps, rendering-first debugging, durable state directories, and content-addressable artifacts for payloads exceeding 4 KB. We specify data contracts, MCP tool semantics, preset evaluation suites for Parts 1 and 2, and quality gates including architectural tests that forbid tool names in Perception prompts.

Index Terms—agentic systems, retrieval-augmented generation, cognitive architecture, Model Context Protocol, FAISS, goal decomposition, vector memory

I. Introduction

Multi-hop question answering with tools and persistent memory requires explicit control over *what* is planned, *what* is retrieved, and *when* a tool is invoked. Monolithic agents often conflate planning, tool selection, and synthesis, which increases looping, redundant fetches, and opaque failures when context is truncated.

The **EAG Cognitive Agent** (Session 7) addresses this by splitting responsibilities across four roles with Pydantic contracts (`schemas.py`). The orchestrator coordinates Memory retrieval, Perception goal lists, Decision outputs (plain answer or exactly one tool call), and Action execution via MCP. Vector memory indexes chunked facts with embeddings from Gateway V7 (`/v1/embed`, Ollama `nomic-embed-text`), enabling semantic search through `search_knowledge` and bulk indexing tools.

This document presents the **system design** required for course submission: high-level flow, role semantics, MCP suite, RAG pipelines, evaluation presets, and runtime layout.

II. Design Principles

TABLE I
DESIGN PRINCIPLES AND ENFORCEMENT

Principle	Enforcement mechanism
Separation of concerns	One responsibility per role; schemas define cross-boundary data
One goal per Decision step	Decision receives only the current goal text, not the full plan
Intent vs. tools	Perception plans in natural-language intent; Decision maps to MCP via docstrings and `decision_system.txt`
Rendering-first debugging	Memory hits expose `chunk:`, `raw:`, `search-results:` before new SYSTEM rules
Durable state	`state/` (course) and `state_jobs/` (Part 2) persist memory and FAISS
Large payloads as artifacts	Responses > 4 KB stored as `art:...`; Memory holds handles

Perception must **never** name MCP tools in goals—an architectural gate verified by `check-perception-gate.sh` and `tests/test_perception_gate.py`.

III. System Architecture

A. High-Level Control Flow

```
User query → Memory.remember(query)
                → LOOP (max 24):
                    Memory.read → top-k hits
                Perception.observe → goals + done flags
                    if all done → exit
                    goal ← first open goal
                attach artifacts if required
                Decision.next_step → answer OR one tool
if tool → Action.execute(MCP) → Memory.record_outcome
                → final_answer_from(history)
```

Fig. 1. Orchestration loop in `agent7.py`.

B. Process Layout

TABLE II
MAJOR COMPONENTS

Component	Module	Function
Orchestrator	`agent7.py`	Loop, CLI presets, MCP session, state directory selection
Memory	`memory.py`	JSON store + FAISS; vector then keyword retrieval
Perception	`perception.py`	Goal decomposition, done flags, artifact attachment
Decision	`decision.py`	Single LLM call: answer or one tool
Action	`action.py`	MCP `call_tool` dispatch
MCP server	`mcp_server.py`	Twelve stdio tools
Gateway	`gateway.py`	Chat + embeddings (V7 preferred)
Vector index	`vector_index.py`	FAISS IndexFlatIP, L2-normalized inner product
Indexing	`indexing.py`	Chunking → `Memory.add_fact`
Prompt render	`prompt_render.py`	Formats hits for Perception/Decision

The agent spawns `mcp_server.py` as a subprocess. Part 2 presets (`j1`–`j5`) set `EAG_STATE_DIR=state_jobs` for an isolated jobs index.

IV. Four-Role Cognitive Model

A. Memory

Memory provides **remember** (classify and store query), **read** (top-k retrieval with vector-first, keyword fallback), **record_outcome** (tool results with embeddings and optional artifact IDs), and **add_fact** (indexing pipeline). Storage comprises `memory.json`, `index.faiss`, and `index_ids.json` (768-dimensional embeddings). Item kinds include `fact`, `preference`, `tool_outcome`, and `scratchpad`. Indexed chunks (`[workspace:…]`, `[corpus:…]`) rank above query-echo scratchpad in mixed hit lists.

TABLE III
TWELVE MCP TOOLS

B. Perception

Perception decomposes the user query into ordered goals, sets `done=true` only when **run history** evidences completion (not from the LLM alone), and assigns `attach_artifact_id` for synthesis goals. Code may seed goals for URL fetch patterns and indexed-paper RAG (presets **f2**, **h**). The role uses Gemini via gateway (`provider=g`) with structured JSON (`PerceptionLLMOutput`).

C. Decision and Action

Decision sees one goal, memory hits, attached artifacts, recent history, and MCP schemas; it returns **exactly one** substantive answer **or** a single tool call per policy in `prompts/decision_system.txt`. Action executes MCP tools without an LLM; large responses become artifacts.

V. MCP Tool Suite

Tool	Purpose
`web_search`	Tavily primary, DuckDuckGo fallback
`fetch_url`	HTTP fetch; Wikipedia via MediaWiki API
`get_time`	Timezone-aware clock
`currency_convert`	FX conversion
`read_file`, `list_dir`, `create_file`, `update_file`, `edit_file`	Workspace file I/O
`index_document`, `index_directory`	Chunk + embed into Memory
`search_knowledge`	Vector search over indexed chunks

Chunking uses ~400-word segments with 80-word overlap; embeddings use gateway `retrieval_document` task type.

VI. Vector Memory and RAG

Course papers (presets e–h): Sources in `papers/` copy to `workspace/papers/`. Preset **e** indexes one document; **f1** bulk-indexes via `index_directory`; **f2** / **g** / **h** query and synthesize or compare across corpora (retain `state/` between **f1** and **f2**).

TABLE IV
PART 1 PRESETS (REPRESENTATIVE)

Preset	Capability
a	`fetch_url` + artifact attach + extract
b	`web_search` + weather + multi-goal synthesis
c1/c2	Reminders + cross-run memory recall
d	Search + read + numbered synthesis
e–h	Indexing + RAG + cross-paper comparison

Part 2 maps **j1–j5** to semantic and factual jobs-market queries (IC modeling, remote ML compensation, early career outside US, max NLP/LLM pay, UK role comparison).

VIII. Data Contracts and Runtime

Schemas: `MemoryItem`, `Goal`, `Observation`, `DecisionOutput` (XOR answer/tool), `PerceptionLLMOutput` (optional `artifact_index` into formatted hits).

Runtime paths: `state/`, `state_jobs/`, `workspace/`, and `logs/` are gitignored and rebuilt per machine. Committed assets include `corpus/ai_jobs/` and `papers/`.

Dependencies: Python 3.11+, uv, LLM Gateway V7 on port 8107, Ollama `nomic-embed-text`, API keys in `.env` (never committed).

Quality gates: `uv run pytest` (26+ unit tests without gateway); `./scripts/check-perception-gate.sh`.

Part 2 — AI jobs market: Kaggle CSV remains local (`data/ai_jobs/raw/`, gitignored). Build scripts produce `corpus/ai_jobs/items/*.md` (1,500 items). Bulk indexing targets `state_jobs/`. CLI presets **q1–q5** and agent presets **j1–j5** exercise semantic, factual, and comparison queries.

VII. Evaluation Presets

IX. Conclusion

The EAG four-role architecture trades a single end-to-end prompt for explicit stages: durable vector memory, intent-only Perception, constrained Decision, and deterministic Action. Session 7 adds RAG over indexed documents and a jobs corpus with isolated state. The design is testable, debuggable via rendered memory hits, and extensible by adding MCP tools with docstrings and Decision policy updates—without violating the Perception tool-name gate.

References

[1] Model Context Protocol Specification, Anthropic, 2024. [Online]. Available: https://modelcontextprotocol.io

[2] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, 2019. (FAISS library used for `IndexFlatIP`.)

[3] A. Vaswani et al., "Attention is all you need," *NeurIPS*, 2017. (Course paper corpus includes Transformer source material.)

[4] School of AI, "EAG V3 Session 7 — Cognitive Agent Assignment," course materials, 2026.

[5] Kaggle, "AI Jobs Market Dataset 2025–2026," used for Part 2 corpus build (CSV local-only).

Manuscript prepared for EAG V3 Session 7 project submission — eag-cognitive-agent